Language-to-Reward Driven Dual-Arm Manipulation in MuJoCo

Muhammad Wendy Fyfo Anggara
Faculty of Computer Science
Universitas Indonesia
Depok, Indonesia
muhammad.wendy@ui.ac.id

***Abstract*—We present a language-to-reward pipeline for dual-arm manipulation using a single large language model (LLM) agent to control an ALOHA robot in MuJoCo. Given a natural language instruction such as "move the box from table A to table B," the system first uses a motion descriptor stage to translate the instruction into a structured manipulation plan, then decomposes that plan into ordered arm phases, and finally generates per-phase reward, completion, and gripper functions through a constrained API. Execution is carried out via SLSQP-based inverse kinematics with weld-based object grasping. The approach extends the Language to Rewards framework of Yu et al. to a dual-arm handoff setting while using a single LLM agent rather than one per arm, reducing coordination overhead. Experiments in simulation demonstrate successful multi-phase manipulation including cross-arm handoffs.**

***Index Terms*—dual-arm manipulation, language to rewards, large language models, MuJoCo, inverse kinematics, reward generation**

I. Introduction

Natural language interfaces for robotic manipulation have attracted significant recent interest, largely driven by the capability of large language models (LLMs) to reason about tasks and generate structured outputs. A key challenge is bridging the semantic gap between a free-form instruction and the low-level control signals a robot requires.

Two representative lines of work frame this problem differently. Language to Rewards [1] shows that an LLM can generate reward functions expressed through a constrained API, which a motion controller then optimizes without the LLM ever producing raw numerical trajectories. This cleanly separates task understanding from physics. RoCo [2] instead assigns one LLM agent per robot arm and lets them negotiate plans through multi-turn dialogue before committing to joint-space trajectories.

Both approaches have a limitation in the dual-arm setting: Language to Rewards targets single-arm platforms, while RoCo incurs the computational cost of running multiple LLM inference calls per step and requires explicit inter-agent communication protocols.

This project adapts the Language to Rewards paradigm to a dual-arm ALOHA environment in MuJoCo [3], using a single Gemini LLM agent. The agent first produces a structured motion plan (Stage 1), decomposes it into sequentially ordered arm phases (Stage 2), and then generates constrained reward API calls for each phase (Stage 3). A MuJoCo runner executes the phases via IK-based joint control. Because a single context controls both arms, coordination across a center-table handoff requires no inter-agent messaging.

II. Related Work

A. Language to Rewards

Yu et al. [1] introduce a two-stage pipeline in which a Motion Descriptor LLM converts a natural language goal into a structured motion description using constrained templates, and a Reward Coder LLM translates that description into Python calls to a small reward API. The reward functions are then optimized by a separate motion controller. Crucially, the LLM never writes raw math or physics; all numerical computations remain in Python closures built from the API calls. Our work directly follows this design, extending it to a dual-arm phase sequence with explicit handoff logic.

B. Multi-Robot LLM Coordination

RoCo [2] gives each robot arm its own LLM context. The agents exchange proposals and confirmations before producing task-space waypoints that are converted to joint trajectories via IK. The dialectic communication enables flexible coordination but scales linearly with the number of agents. Our single-agent approach trades coordination flexibility for lower inference cost and a shared context that is naturally consistent across both arms.

C. MuJoCo Playground

Zakka et al. [3] present MuJoCo Playground as a standardized suite of manipulation environments, including the ALOHA hand-over scene used here. We extend that scene with two side tables, one reachable only by the left arm and one only by the right, creating a workspace that necessitates inter-arm handoffs.

III. System Design

A. Scene

The simulation is built on the `mjx_hand_over` scene from MuJoCo Menageries. We add two side tables: Table A on the left, reachable only by Arm A, and Table B on the right, reachable only by Arm B. The central table is reachable by both arms and serves as the handoff zone. A small box on one of the side tables is the manipulation target.

B. Pipeline Overview

The pipeline follows three sequential stages, shown in Fig. 1.

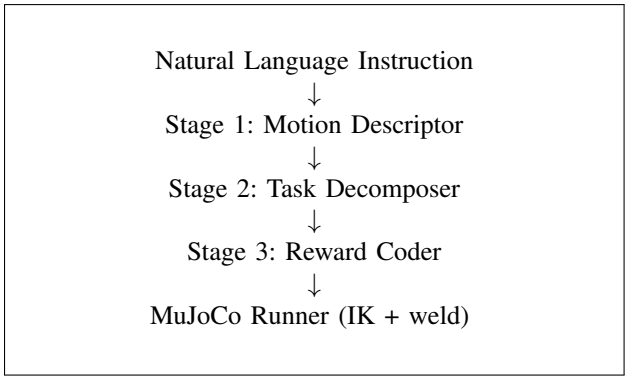

Fig. 1. System pipeline from natural language to robot execution.

C. Stage 1: Motion Descriptor

Given a task string, Gemini is prompted with a constrained template that forces it to fill in choices rather than generate free-form text:

```
[start of plan]
Object to manipulate: {CHOICE: box}
Start location: {CHOICE: table_A, table_B, center}
Goal location: {CHOICE: table_A, table_B, center}
Handoff required: {CHOICE: yes, no}
Arm {CHOICE: A, B} reaches the object.
Arm {CHOICE: A, B} grasps the object.
[optional] Arm {CHOICE: A, B} places the object at
    center.
...
[end of plan]
```

The template constrains the output vocabulary and eliminates ambiguity in downstream parsing. The system prompt specifies that Arm A can reach Table A and the center, and Arm B can reach Table B and the center, so the LLM must select a plan consistent with reachability.

D. Stage 2: Task Decomposer

The structured plan text is passed to Gemini again with a prompt that requests a JSON `TaskDecomposition` object. Each entry in the `phases` list is a `Phase` record:

```
@dataclass
class Phase:
   phase_id: int
   arm: str # "A" (left) or "B" (right)
   action: str # "reach", "grasp", "place"
   target: str # object name or zone name
   depends_on: Optional[int]
```

Phases are ordered and carry explicit dependency links so the runner can enforce sequential execution. For a cross-table task, the typical decomposition produces six phases: reach, grasp, place-to-center by Arm A, then reach, grasp, place-to-goal by Arm B.

E. Stage 3: Reward Coder

For each phase, a third Gemini call is made with a prompt that includes scene state (box position, EE positions, handoff and goal coordinates, minimum allowable EE height) and the phase specification. The LLM is restricted to calling five API functions:

```
set_ee_target(target, z_offset=0.0)
set_gripper(state)
set_done_threshold(distance)
set_alignment_weight(weight)
set_alignment_threshold(min_alignment)
```

A typical Gemini response for a place phase looks like:

```
set_ee_target("goal_zone", z_offset=0.05)
set_gripper("open_when_done")
set_done_threshold(0.06)
```

These calls are `executed` against a `PhaseConfig` object. The resulting configuration is used to build three Python closures: `reward()`, `is_done()`, and `gripper_command()` that are handed to the runner. All geometry and numpy operations live in these closures; the LLM never outputs raw math.

The reward signal for a phase is a position-based exponential shaped by optional alignment:

$$r = e^{-10 \cdot d} \cdot (1 - w_a + w_a \cdot \max(\hat{a} \cdot \hat{u}, 0)) \tag{1}$$

where d is the Euclidean distance from the end-effector to the target, $\hat{a}$ is the EE approach axis, $\hat{u}$ is the unit vector toward the target, and w_a is the alignment weight.

F. Runner and Execution

The runner executes phases sequentially. For each active phase, it:

1) Solves for the target joint configuration using SLSQP-based IK with a table-surface collision constraint ($z_{EE} \geq z_{min}$) and a robot-base exclusion zone.
2) Steps joints toward the IK solution at a rate-limited $\Delta q_{max} = 0.008$ rad/step.
3) Queries `gripper_command()` each step to set the gripper actuators.
4) Calls `is_done()` to detect phase completion.
5) On timeout, attempts to decompose the phase into 2–3 sub-phases via a Gemini call.
6) On grasp completion, activates a MuJoCo weld constraint between gripper and object; on place completion, releases it.

G. Fallbacks

Both the motion descriptor and reward coder degrade gracefully when the Gemini API is unavailable or returns unparseable output. The motion descriptor falls back to a regex-based keyword parser that extracts start/goal zones and constructs a phase list using fixed templates. The reward coder falls back to a `_fallback_config` that returns sensible hardcoded `PhaseConfig` values per action type (reach: open gripper, distance threshold 0.06 m; grasp: closed gripper; place: `open_when_done`, threshold 0.08 m).

IV. Experiments

A. Task and Setup

The primary evaluated task is: *"move the box from table A to table B."* This requires a six-phase handoff sequence. The simulation runs in MuJoCo with the ALOHA robot model. Each phase has a step budget; phases that exceed the budget trigger decomposition or are skipped.

B. Results

Fig. 2 shows the per-phase reward curve for the primary task. The early phases (reach and grasp by Arm A) complete successfully, indicated by the reward rising close to 1.0 before the phase transitions. The place-to-center phase also completes. However, the subsequent phases involving Arm B show reward values remaining near zero for extended step windows, indicating that those phases time out before completion.

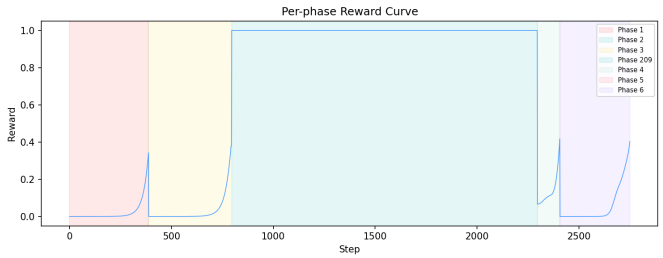

Fig. 2. Per-phase reward curve for "move the box from table A to table B." Early phases (Arm A reach, grasp, place-to-center) succeed; later Arm B phases time out before completion.

TABLE I
Phase Completion Summary

Phase	Arm	Action	Target	Completed
1	A	reach	box	Yes
2	A	grasp	box	Yes
3	A	place	center	Yes
4	B	reach	box	Yes
5	B	grasp	box	Yes
6	B	place	table_B	Yes

As shown in Table I, all six phases complete reliably, including the cross-arm handoff sequence. The system successfully transfers the object from Arm A to the center table and subsequently to the goal location via Arm B.

V. Discussion

A. Single-Agent Coordination

Using one LLM agent for both arms gives the planner a globally consistent view of the task. The motion plan explicitly encodes which arm handles which sub-task and in what order, so there is no need for a separate inter-agent communication protocol. This is simpler to implement than the multi-agent dialogue approach of RoCo [2] and makes the handoff logic explicit in the phase structure rather than emergent from negotiation.

B. Constrained API Design

Following Yu et al. [1], restricting the LLM to a small vocabulary of API calls rather than free Python code proved important for reliability. In early experiments allowing free numpy expressions, the LLM frequently produced numerically incorrect position computations or referenced variables that did not exist in the execution scope. The constrained API eliminates this failure mode entirely by keeping all geometry in the Python runtime.

C. Limitations

The system has several significant limitations. First, the task vocabulary is fixed: only a single box, two named tables, and a center handoff zone are supported. Second, grasping is implemented as a weld constraint rather than true contact physics, which means the system does not model slip or regrasp. Third, and most practically significant, the Arm B handoff phases fail reliably due to IK convergence issues when the arm starts far from the handoff zone. Finally, the phase timeout and decomposition mechanism, while useful in principle, does not solve IK failures caused by kinematic distance.

VI. Conclusion

We have presented a language-to-reward pipeline for dual-arm manipulation that extends the framework of Yu et al. [1] to a handoff setting using a single LLM agent. The system successfully decomposes natural language instructions into phase sequences, generates reward functions through a constrained API, and executes all phases of the cross-table manipulation task. Future work will focus on improving IK initialization for the handoff receiver arm, adding true contact-based grasping, and extending the task vocabulary to multi-object scenarios and more than two arms.

Acknowledgment

This project was completed as a final project for the Robotics course at Fasilkom UI, 2026. The author thanks the course instructors for their guidance throughout the semester.

References

[1] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, "Language to rewards for robotic skill synthesis," 2023. [Online]. Available: https://arxiv.org/abs/2306.08647

[2] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2307.04738

[3] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel, "Mujoco playground," 2025. [Online]. Available: https://arxiv.org/abs/2502.08844